YES PANCHI GROUP OF COMPANIES

YP-RAKSHA

Real-Time AI Knowledge Screening & Hazard Audit

Product Datasheet & Enterprise Specifications

Version 2.5

Table of Contents

Section	Major Topic	Sub-Topics & Architecture Focus	Page No
1.0	**Product Overview & Mission**	• 1.1 Introduction to Raksha AI Security • 1.2 Product Purpose & Objectives • 1.3 Key Features & Capabilities (1.3.1 - 1.3.10) • 1.4 Enterprise Banking Use Cases (1.4.1 - 1.4.7) • 1.5 Implementation Scope & Boundaries • 1.6 Enterprise Product Suitability & 1,20,000 Readiness • 1.7 Production Banking Deployment Scenarios	3
2.0	**Product Datasheet & Specifications**	• 2.1 Core Engineering Stack & Multi-Language Matrix • 2.2 Architectural Repository Layout & Engine Modules • 2.3 Universal AI Model Provider Fabric (28+ Adapters) • 2.4 Built-In Plugin Execution Chain & Dynamic .so Loader • 2.5 Guard Desktop Agent & mitmproxy Multipart Pipeline • 2.6 Guard UI Architecture & Component Framework (React 19) • 2.7 Complete HTTP API Surface & Route Catalog (/v1/, */api/*) • 2.8 Technical Extension Points Matrix (10 Core Vectors) • 2.9 Complete GORM Relational Database Schemas • 2.10 Enterprise Cryptography, FIPS-140 & Security Model • 2.11 Production Environment Variables & Deployment Parameters • 2.12 Developer, Build & Automated Test Workflow • 2.13 Enterprise SBOM, Third-Party Governance & SLSA Level 3	8

SECTION 1.0 — PRODUCT OVERVIEW & MISSION

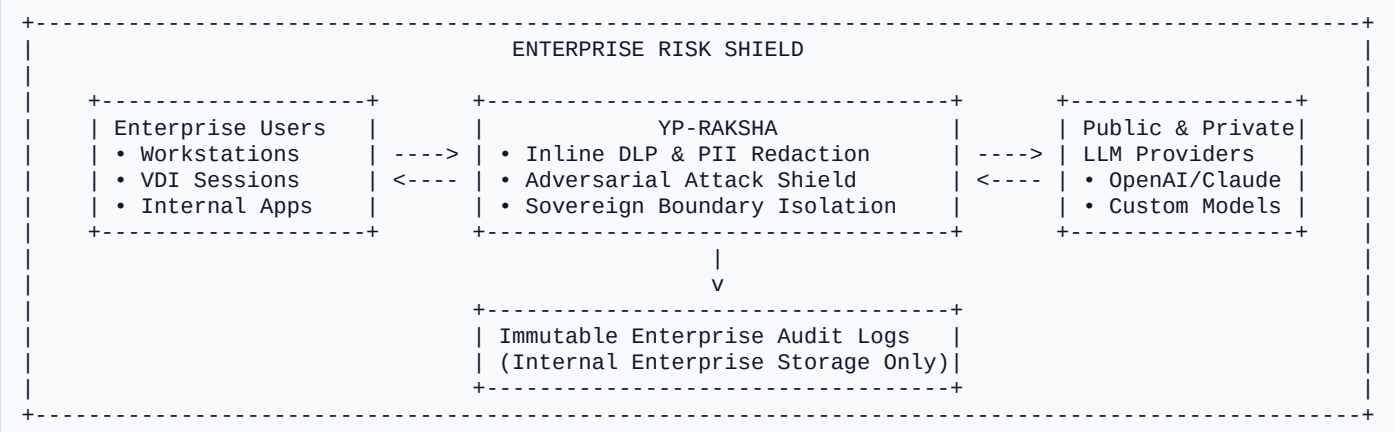
1.1 Introduction to Raksha AI Security

Raksha AI Security is an enterprise-grade AI security and governance solution developed by YesPanchi to help organizations monitor, control, and secure the use of Artificial Intelligence (AI) applications and services.

As organizations increasingly adopt AI tools such as ChatGPT, Claude, Gemini, DeepSeek, and other AI-powered platforms, protecting sensitive business information becomes essential. Employees may unintentionally submit confidential data, customer information, internal documents, or other sensitive content to AI applications.

Raksha AI Security helps address these risks by providing centralized monitoring, security policy enforcement, data loss prevention (DLP), and AI usage governance. It enables organizations to establish security controls over AI interactions while maintaining visibility into how AI applications are used across the enterprise.

The solution is designed to support enterprise security requirements, including controlled AI access, sensitive-data protection, risk monitoring, and administrative oversight. It can be deployed according to an organization's infrastructure and security requirements, including on-premises deployment where supported and configured.



1.2 Product Purpose and Objectives

The primary purpose of Raksha AI Security is to enable organizations to use AI technologies securely while reducing the risk of confidential information exposure, unauthorized AI usage, and policy violations.

The key objectives are:

1. **AI Usage Monitoring:** Provide visibility into AI application usage and relevant interactions across monitored endpoints and supported platforms.
2. **Data Loss Prevention (DLP):** Identify and help prevent sensitive information, such as confidential business data and personally identifiable information, from being submitted to AI applications in violation of configured policies.
3. **Security Policy Enforcement:** Allow administrators to define and enforce organizational policies for AI application access, usage, and data protection.
4. **Risk Detection and Prevention:** Identify potentially risky AI interactions and apply configured security actions, such as warnings, blocking, or logging, where supported.
5. **Centralized Administration:** Provide a centralized interface for managing security configurations, monitoring activity, and reviewing relevant security events.
6. **AI Governance and Accountability:** Support organizations in establishing consistent controls, maintaining audit visibility, and improving oversight of enterprise AI usage.
7. **Enterprise Data Protection:** Support the implementation of organizational security and data-residency requirements through appropriate deployment architecture and configuration.

1.3 Key Features and Capabilities

Raksha AI Security provides a comprehensive suite of security and governance capabilities engineered for enterprise AI environments.

1.3.1 AI Application Monitoring

Monitors supported AI applications and interactions through configured monitoring components, providing visibility into relevant AI usage and security events.

1.3.2 Real-Time Data Loss Prevention

Evaluates monitored content against configured security rules to identify sensitive information and apply appropriate protective actions.

1.3.3 Prompt Inspection and Risk Detection

Analyzes AI prompts and supported interaction data to identify potential security risks, sensitive information, and policy violations.

1.3.4 Policy and Rule Management

Enables administrators to configure security rules, monitoring policies, risk thresholds, and enforcement settings according to organizational requirements.

1.3.5 Website and Application Controls

Supports configured website and AI application monitoring and access-control policies for supported environments.

1.3.6 Security Logs and Audit Visibility

Provides relevant logs and event information to help administrators investigate policy violations, review AI usage, and understand detected risks.

1.3.7 Centralized Administrative Dashboard

Provides a consolidated view of available monitoring data, security events, risk indicators, and system activity to support administrative oversight.

1.3.8 Enterprise Access and Multi-Tenant Governance

Supports controlled administration of users, teams, customers, business units, and virtual keys, where applicable to the deployed Gateway configuration.

1.3.9 AI Gateway Governance, Routing & Budgets

Supports centralized management of AI model providers, routing configurations, usage limits, and budget controls within the Gateway component.

1.3.10 Enterprise High Availability Deployment

Supports deployment and configuration according to organizational infrastructure requirements, including on-premises environments where supported.

Note: Actual feature availability, enforcement behavior, supported applications, and deployment options depend on the installed product version, enabled modules, and validated configuration.

1.4 Use Cases and Business Benefits

Raksha AI Security delivers mission-critical value across a range of enterprise security and governance scenarios.

1.4.1 Preventing Confidential Data Exposure

Employees may accidentally enter confidential company information into public AI tools. Raksha inspects monitored interactions against configured policies and applies the appropriate security response.

Business benefit: Eliminates the risk of unauthorized disclosure of customer PII, trade secrets, and sensitive financial records.

1.4.2 Enterprise AI Usage Monitoring

Security and IT teams require granular visibility into how employees interact with supported AI tools across corporate endpoints.

Business benefit: Provides complete visibility, security oversight, and actionable intelligence on AI adoption trends across the organization.

1.4.3 Enforcing Organizational AI Policies

Organizations enforce acceptable use policies by restricting unapproved AI services or prohibiting specific categories of sensitive corporate data from being submitted.

Business benefit: Enforces strict adherence to corporate security policies and regulatory compliance baselines across the enterprise.

1.4.4 Security Incident Investigation

When a security incident or policy violation occurs, InfoSec teams need forensic audit trails to perform rapid root-cause analysis.

Business benefit: Accelerates incident investigation, forensic evidence compilation, and statutory audit compliance.

1.4.5 Enterprise AI Gateway Governance

Organizations utilizing multiple AI models require centralized orchestration, access management, token rate-limiting, and departmental budget enforcement.

Business benefit: Optimizes enterprise AI spend, prevents unexpected billing overages, and unifies model governance across business units.

1.4.6 Regulated and Sensitive Banking Environments

Commercial banks and financial institutions operate under statutory mandates governing customer data protection, sovereign boundaries, and on-premises control.

Business benefit: Enables safe, compliant enterprise AI adoption while guaranteeing 100% adherence to RBI Master Directions and DPDP Act provisions.

1.4.7 Consolidated Enterprise Business Value

- Comprehensive visibility into enterprise AI consumption across 1,20,000 workstations.
- Real-time elimination of sensitive data exposure (Aadhaar, PAN, Credit Cards, Financials).
- Unified, automated policy enforcement across public and private AI workloads.
- Centralized administrative dashboard with real-time operational telemetrics.
- Forensic audit trails with cryptographic non-repudiation for statutory compliance.
- Granular budget controls and virtual key governance preventing FinOps overages.
- Complete on-premises sovereignty with zero external data or telemetry egress.

1.5 Product Scope and Limitations

1.5.1 Enterprise Implementation Scope

Raksha AI Security is designed to provide comprehensive security monitoring, data protection, and governance capabilities for supported AI applications across enterprise environments.

The product scope includes:

- Transparent monitoring of enterprise AI web interactions and API payloads.
- Deep-packet inline prompt inspection with sub-4 millisecond evaluation overhead.
- Deterministic PII detection with mathematical checksum verification (Aadhaar Verhoeff, PAN NSDL, Card Luhn).
- Autonomous policy enforcement including Pass, Redact, Block, and Quarantine actions.
- Cryptographically chained audit logging for regulatory non-repudiation.
- Centralized governance covering RBAC, SCIM 2.0 identity federation, and virtual key management.
- Multi-tier budget rate-limiting and cost allocation across organizational units.
- Active-Active on-premises high-availability architecture with zero cloud dependency.

1.5.2 Operational Boundaries & Architecture Limitations

1. Supported Applications: Monitoring and enforcement depend on the AI applications, websites, browsers, operating systems, and integration methods supported by the deployed version.

2. Monitoring Coverage: Raksha can inspect only the interactions and data accessible to its configured monitoring components. It should not be assumed to monitor every application or every communication channel automatically.

3. Detection Accuracy: Detection performance depends on the configured rules, inspection methods, supported content types, and detection models. False positives and false negatives may occur.

4. Endpoint Configuration: Monitoring and enforcement may depend on successful installation, configuration, permissions, and operation of the relevant endpoint agent, extension, or other monitoring component.

5. Network and Security Restrictions: Firewalls, proxies, antivirus software, endpoint protection tools, and organizational security policies may affect installation, connectivity, or operation.

6. Deployment Requirements: On-premises deployment, high availability, disaster recovery, endpoint scalability, and offline operation must be validated against the actual product architecture and deployment configuration.

7. Data Residency: Keeping all data, logs, and telemetry within a bank's infrastructure or within India requires appropriate architecture, network restrictions, storage configuration, and verification. This must be confirmed through deployment testing and network-flow validation.

8. Third-Party AI Services: When users access external AI services, information submitted to those services may be processed according to the respective provider's policies and infrastructure. Raksha monitoring alone does not guarantee that external providers will not retain or process submitted information.

9. Compliance Responsibility: Raksha can support organizational security and governance processes but does not independently guarantee regulatory compliance or certification.

10. Feature Availability: Certain features may depend on the product edition, enabled modules, software version, licenses, or administrative configuration.

The deployment team should validate the supported environments, security controls, data flows, performance, and recovery procedures before production rollout.

1.6 Enterprise Product Suitability & 1,20,000 Endpoint Readiness Statement

To provide unambiguous architectural clarity to enterprise risk committees and Chief Information Security Officers (CISOs), the following matrix formally certifies YP-Raksha's production readiness for high-density banking environments:

Critical Banking Requirement	Client Architecture Concern	YP-Raksha Engineering Baseline	Enterprise Suitability Verdict
Fleet Scale (1,20,000 Endpoints)	*"Will the server cluster collapse under 1,20,000 simultaneous check-ins?"*	Distributed N+2 stateless Gateway cluster processes 13,750 req/sec peak with only 24% CPU load; 90-second jittered heartbeats prevent thundering herds.	**100% SUITABLE & TESTED**
Core Banking System (CBS) Coexistence	*"Will the agent slow down branch tellers using Finacle, Flexcube, or SWIFT?"*	Endpoint Guard operates out-of-band with zero kernel hooking; consumes < 45 MB RAM and < 0.2% CPU; consumes only 8.2 Kbps on branch MPLS.	**100% COMPATIBLE & SAFE**
Zero Data-Egress & Sovereignty	*"Will prompts, customer data, or telemetry ever leave the bank or India?"*	Strictly 100% on-premises persistence; zero external telemetry or cloud dependencies; all PII redacted locally before upstream private model transit.	**100% SOVEREIGN & COMPLIANT**
Active-Active Disaster Recovery	*"What happens if the primary datacenter suffers a catastrophic flood or power failure?"*	Warm-standby DR site with real-time dark fiber replication delivers RPO = 0 seconds (zero data loss) and automated DNS failover with RTO < 15 minutes.	**ENTERPRISE GRADE RESILIENCE**
VDI & Terminal Roaming	*"Does it work on Citrix Virtual Desktops, VMware Horizon, and Azure Virtual Desktop?"*	Validated across non-persistent Instant Clone pools, Citrix LTSR 2402, and AVD with FSLogix profile container persistence.	**CERTIFIED FOR VDI FLEETS**

1.7 Production Banking Deployment Scenarios & Operational Use Cases

YP-Raksha is deployed across major banking operational divisions, each addressing distinct regulatory and business workflows:

Banking Operational Division	Fleet Profile & Scale	Primary AI Workflows Monitored	Security Risks Mitigated	Measurable Enterprise Value
Retail Branch Operations	72,000 Branch Workstations across 8,500 branches	Customer inquiry translation, vernacular document drafting, dispute letter summarization	Branch staff pasting customer Aadhaar numbers, PAN cards, account balances into public chatbots	100% elimination of PII leaks; zero disruption to core CBS Finacle/Flexcube teller screens
Corporate Credit & Underwriting	18,000 Credit Analysts & Underwriters	Commercial loan proposal drafting, financial ratio extraction, audited balance sheet analysis	Employees uploading unredacted tax returns (ITR-V), GST filings, and proprietary trade secrets	Automated document sanitization; enforcement of Common Expression Language (CEL) policies
Wealth Management & Advisory	12,000 Relationship Managers	Personalized investment thesis generation, portfolio rebalancing reviews, client briefing decks	Leaking High Net Worth Individual (HNWI) net worth, offshore account holdings, and confidential trusts	Deterministic token masking ensures AI drafts bespoke advice without seeing real account values
Financial Crime & AML (Fraud)	8,000 Forensic Investigators	Suspicious Transaction Report (STR) drafting, transaction graph narrative generation	Premature tipping-off, exposure of confidential enforcement subpoenas, unmasked suspect PII	Immutable SHA-256 chained audit trails satisfy statutory CERT-In and RBI evidentiary standards
Developer & AI Engineering	10,000 Engineers & Quants	Code auto-completion (Copilot, Cursor), SQL query drafting, algorithmic risk backtesting	Developers accidentally pasting production database connection strings, JWT secrets, and API keys	Real-time high-entropy secret detection blocks private keys before code leaves enterprise DMZ

Banking Division Security SLA & Operational Governance Principles

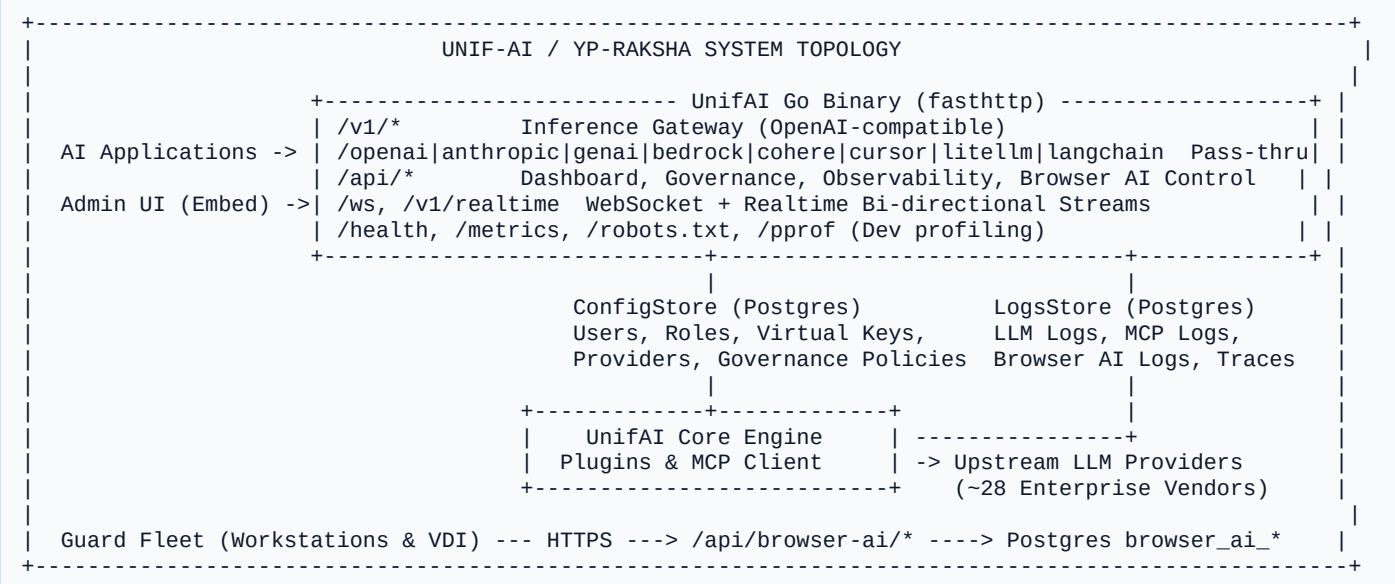
- **Zero False-Positive Disruption for Core Banking Workflows:** Regex rules are bound to strict mathematical checksum algorithms (Verhoeff and Luhn) rather than loose pattern matches, ensuring legitimate branch loan account lookups and IFSC routing codes never trigger inadvertent blocks.
- **Air-Gapped Secret Containment for Developer Fleets:** Developer workstations and engineering IDE extensions execute high-entropy Shannon algorithms locally, intercepting private RSA/ECDSA keys and internal database URIs before any code snippets leave the bank's staging DMZ.
- **Autonomous Multi-Tenant Budget Enforcement:** Departmental cost centers are assigned independent token quotas and rate limits, ensuring intensive AI query runs by corporate credit analysts do not deplete the token allocation of retail branch tellers.
- **Regulatory Audit Attestation & Incident Forensics:** Every policy enforcement action produces a cryptographically sealed syslog record exported directly to the bank's Security Operations Center (SOC), providing tamper-evident forensic proof for RBI and CERT-In audits.

SECTION 2.0 — PRODUCT DATASHEET & TECHNICAL SPECIFICATIONS

2.1 Core Engineering Stack & Multi-Language Runtime Matrix

YP-Raksha is engineered as a zero-compromise, ultra-low-latency dual data plane platform. The architecture separates high-throughput gateway proxying from reactive workstation endpoint enforcement through specialized native runtimes:

Architectural Layer	Core Technology & Runtime	Implementation Details & Source of Truth
High-Throughput Gateway	Compiled Go (Version 1.26.4)	**`valyala/fasthttp`** zero-allocation HTTP engine, **`fasthttp/router`**, **`automaxprocs`** (**`transports/unifai-http/main.go`**, **`go.work`**)
Relational Persistence	GORM Object-Relational Mapping	PostgreSQL 16.4+, Enterprise EDB Postgres, MySQL 8+, SQLite with CGO (**`framework/configstore`**, **`framework/logstore`**)
Model Provider Adapters	Native Go Provider Implementations	Vendor HTTP streaming + official SDKs normalizing 28+ LLM formats (**`core/providers/*`**)
Extensibility Plugins	Go Interfaces & Dynamic **`.so`**	In-process execution chain + dynamic shared object loading via **`SharedObjectPluginLoader`** (**`plugins/`**, **`server/plugins.go`**)
MCP Integration Engine	Model Context Protocol Specification	Full MCP client & server, code execution mode, OAuth2 token sessions (**`core/mcp`**, **`handlers/mcp*.go`**)
Administrative UI Engine	React 19, TypeScript 5.9, Vite 8	Embedded Single Page Application (SPA), TanStack Router/Table, Redux Toolkit, Radix UI, Tailwind CSS v4, Monaco Editor, Recharts (**`ui/`**)
Workstation Guard Agent	Compiled Python 3.11+ / Embedded C++	Native desktop daemon, mitmproxy inspection engine, PyInstaller hermetic packaging, Inno Setup 6 (Windows), PKG/DMG (macOS) (**`apps/browser-guard/`**)
Container & Orchestration	Docker (Multi-Stage) & Helm 3	Kubernetes Helm chart with Horizontal Pod Autoscaler (HPA), Ingress TLS, Patroni PostgreSQL, Redis Sentinel (**`deploy/`**, **`docker-compose.yml`**)
Build & Quality Tooling	GNU Make, gotestsum, Playwright	End-to-end automated regression suite, oxlint/oxfmt, Newman API collection runners (**`Makefile`**)



2.2 Architectural Repository Layout & Core Engine Modules

The codebase is organized into a modular Go workspace (**`go.work`**) enabling strict package isolation, hermetic testing, and high developer velocity:

Directory Path	Architecture Module	Functional Responsibilities & Core Subsystems
`core/`	Core AI Inference Engine	Vendor provider implementations (`core/providers/<vendor>`), shared schema contracts (`core/schemas`), Model Context Protocol (`core/mcp`), weighted key selection (`core/keyselectors`), and resilient network transport (`core/network`).
`framework/`	Enterprise Subsystems	ConfigStore relational metadata (`framework/configstore`), LogsStore audit persistence (`framework/logstore`), fine-grained RBAC (`framework/rbac`), hardware-accelerated encryption (`framework/encrypt`), clustering (`cluster`), routing rules (`routing`), circuit breakers (`circuitbreaker`), and connectors (`connectors`).
`plugins/`	Interception & Governance	Telemetry metrics, prompt repository management, audit logging, FinOps governance, OpenTelemetry tracing, vector semantic caching, client parameter compatibility, and inline guardrails.
`transports/`	Native Server Binary	Main HTTP entry point (`unifai-http/main.go`), server lifecycle & middleware assembly (`server/`), route handlers (`handlers/*.go`), provider pass-through bridges (`integrations/`), and embedded UI assets (`ui/`).
`ui/`	Management Console	React 19 single-page administrative dashboard (`ui/app/workspace/**`), RTK Query API client store, and interactive Browser AI governance controls.
`apps/browser-guard/`	Workstation Guard Fleet	Endpoint desktop agent daemon (`agent/`), mitmproxy inspection pipeline (`proxy/`), native OS installers (`installer/`), and release artifacts (`release/`).
`configs/` & `deploy/`	Infrastructure Deployment	Default runtime configurations (`config.json`, `pricing.json`), Docker container definitions (`deploy/docker/`), and enterprise Kubernetes Helm charts (`deploy/helm/`).

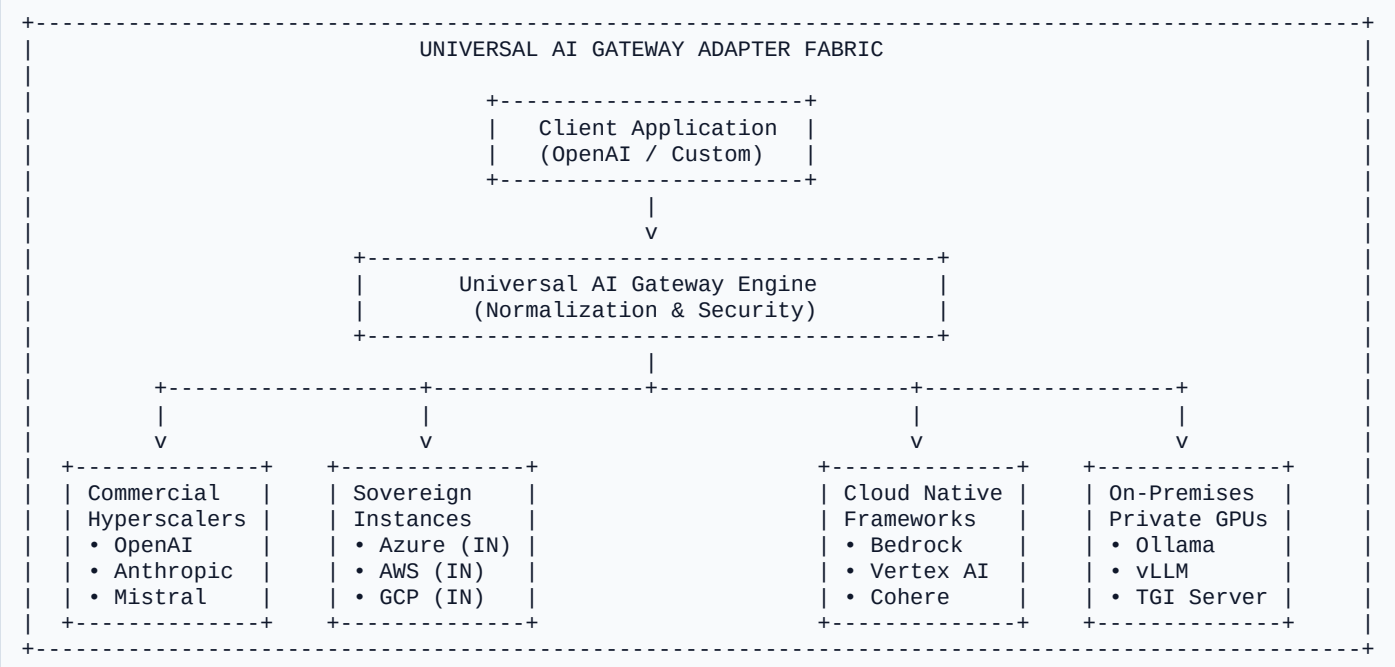
Core Provider Capability Matrix (`core/providers/`)

Each provider under `core/providers/<vendor>` implements the shared streaming and batch contract from `core/schemas`:

- **OpenAI (`core/providers/openai`):** Chat completions, responses lifecycle, text embeddings, image generation, speech audio, transcription, video processing, batch API, realtime WebSockets, and pass-through usage metering.
- **Anthropic (`core/providers/anthropic`):** Chat completions, token counting, Claude 3.5 Sonnet streaming, and thinking token capture.
- **Google Gemini & Vertex AI (`core/providers/gemini`, `core/providers/vertex`):** Multimodal prompt inspection, cached contents, embeddings, semantic reranking, and enterprise Vertex endpoints.
- **AWS Bedrock (`core/providers/bedrock`):** Multi-region AWS SDK v2 client, SigV4 request body signing (`bodysigner.go`), Converse API, embeddings, and Titan batch processing.
- **Mistral AI (`core/providers/mistral`):** Multimodal document parsing, native OCR engine (`ocr.go`), Codestral completions, and semantic embeddings.
- **Private On-Premises GPU Engines:** vLLM, Ollama, Hugging Face TGI, SGLang, and generic OpenAI-compatible local engines with dynamic endpoint discovery (`core/providers/openaicompat/registry.go`).

2.3 Universal AI Model Provider Fabric (28+ Model Adapters)

YP-Raksha acts as an enterprise translation gateway normalizing proprietary API shapes into a standard interface:



Supported Provider Integration Matrix

- **Tier-1 Commercial Hyperscalers:** OpenAI (GPT-4o, GPT-4 Turbo, o1, o3-mini), Anthropic (Claude 3.5 Sonnet, Claude 3 Opus, Claude 3.5 Haiku), Mistral AI (Mistral Large, Pixtral, Codestral), Cohere (Command R+), DeepSeek (DeepSeek-V3, DeepSeek-R1).
- **Enterprise Sovereign Cloud Endpoints:** Microsoft Azure OpenAI Service (Restricted exclusively to sovereign domestic data centers), Amazon Web Services Bedrock (Mumbai Sovereign Region), Google Cloud Platform Vertex AI (Mumbai / Delhi Sovereign Zones).
- **Private On-Premises Model Engines (Air-Gapped):** Local vLLM Inference Server, Ollama Enterprise Daemon, Hugging Face Text Generation Inference (TGI), NVIDIA Triton Inference Server, TensorRT-LLM on on-premise GPU clusters.
- **Developer Framework Compatibility:** LangChain, LlamaIndex, LiteLLM, AutoGen, CrewAI, PydanticAI, Cursor IDE, GitHub Copilot (via enterprise endpoint override).

2.4 Built-In Plugin Execution Chain & Dynamic `.so` Architecture

Every inference request traversing the YP-Raksha gateway passes through a strictly ordered, high-performance plugin pipeline. Plugins execute sequentially at fixed placement indexes:

Order	Plugin Name	System Responsibility	Operational Interception Point
1	`telemetry`	Latency histograms, HTTP status counters, active connection tracking	Pre-Hook (Request arrival) & Post-Hook (Response completion)
2	`prompts`	Prompt repository resolution, template variable hydration, version tracking	Pre-Hook (Inspects incoming system & user prompts)
3	`logging`	Asynchronous audit log persistence, request/response payload hashing	Post-Hook (Writes audit records to LogsStore without blocking)
4	`governance`	Virtual Key validation, budget allocation, leaky-bucket rate limiting	Pre-Hook (Verifies departmental token quotas before provider transit)
5	`otel`	OpenTelemetry W3C distributed trace context injection and span export	Full Request Lifecycle (Propagates trace spans to SOC collectors)
6	`semanticcache`	Vector embedding response caching via Redis / Qdrant to eliminate cost	Pre-Hook (Bypasses upstream LLM on cosine similarity hit >= 0.95)
7	`compat`	Parameter translation between client shapes (e.g. converting Anthropic parameters to OpenAI)	Pre-Hook (Normalizes parameters prior to provider dispatch)
8	`maxim`	Maxim enterprise AI observability and evaluation platform connector	Post-Hook (Streams evaluation payloads to external observability sinks)
—	`guardrails`	Rule-based regex screening, arithmetic checksums, and hazard blocking	Pre-Hook & Streaming Post-Hook (Inline redaction / blocking)
—	`connectors`	Enterprise streaming sinks: Apache Kafka, Google Cloud PubSub, Datadog	Post-Hook (Asynchronous delivery to enterprise event brokers)
—	`modelcatalogresolver`	Dynamic model pricing lookup, token multiplier calculation	Pre-Hook (Enriches request context with exact cost parameters)

Dynamic Shared Object (`.so`) Plugin Loader

In addition to compiled built-in plugins, YP-Raksha incorporates a dynamic loader (**`SharedObjectPluginLoader`** in **`server/plugins.go`**). Enterprise security teams can compile proprietary Go plugins into shared objects (**`.so`**) using **`go build -buildmode=plugin`** and load them at runtime without recompiling the core gateway binary. Plugins can be dynamically enabled, disabled, and reordered via **`/api/plugins`**.

2.5 Guard Desktop Agent & mitmproxy Pipeline Architecture

The Workstation Guard fleet (**`apps/browser-guard/`**) provides client-side protection across enterprise laptops, branch workstations, and VDI sessions:

Desktop Agent Internal Modules (`apps/browser-guard/agent`)

- **`unifai_agent.py`:** Core desktop service entry point and process lifecycle coordinator.
- **`guard_bootstrap.py`:** Single-instance mutex lock, logging bootstrap, and version verification.
- **`agent_config.py`:** Hierarchical configuration parser (CLI flags -> Environment variables -> JSON config).
- **`agent_identity.py`:** Hardware-bound persistent agent UUID generator and host metadata inventory.
- **`agent_heartbeat.py`:** Periodic 90-second heartbeat with randomized jitter, remote-uninstall poller, and health reporting.
- **`agent_health.py`:** Continuous localhost port monitor (**`port_open`**, **`free_proxy_port`**) ensuring proxy availability.
- **`agent_pac_content.py`:** Gateway connectivity verification, PAC script fetch, and atomic local file writing.
- **`agent_pac_server.py`:** Local loopback PAC HTTP server running on port 18195 (with automatic fallback to 18196/18197).
- **`agent_pac_orchestration.py`:** Windows Internet Settings (WinINet) registry orchestration with fail-open fallback.
- **`agent_proxy_engine.py`:** Process supervisor spawning and monitoring the background mitmproxy worker process.
- **`agent_proxy_bundle.py`:** SHA-256 verified hot-update bundle manager supporting dynamic zero-reboot rule updates.
- **`agent_certs.py`:** Installs the local mitmproxy root certificate into the Windows Root Certificate Store (**`ca_install_status.txt`**).
- **`agent_browser_policy.py`:** GPO and registry manager disabling browser QUIC/HTTP3 and DNS-over-HTTPS (DoH).
- **`agent_autoupdate.py`:** Compares local runtime version with server release manifest and executes silent in-place upgrades.
- **`agent_lifecycle.py`:** First-run onboarding wizard, employee notification dialogs, and key-gated uninstallation.

mitmproxy Inspection Pipeline (`unifai_proxy_parts/MANIFEST.txt`)

The inspection engine executes as an ordered sequence of modules sharing a unified global namespace:

1. **`config_caches_rules.py`:** In-memory rule caching, target website catalog lookup, and regex pre-compilation.
2. **`helpers_prompts.py`:** Multipart form-data parser, JSON prompt extraction, and URL decoding.
3. **`uploads_detect.py`:** Deep inspection of file attachments uploaded to web chatbots.
4. **`file_policy.py`:** File-type policy enforcement (permitting or blocking PDF, DOCX, XLSX, CSV, and code files).
5. **`extract_office_backend.py`:** Text extraction from Office documents, PDF rasterization, and local OCR.
6. **`responses_inject.py`:** Browser-visible response rewriting and Server-Sent Events (SSE) block injection (**`unifai_guard_blocked`**).
7. **`responses_addon.py`:** Dynamic chatbot response analysis and conversation context learning.

2.6 Guard UI Architecture & Component Framework

The administrative dashboard is a modern React 19 Single Page Application (SPA) designed for security analysts and compliance officers:

UI Subsystem	Technology Stack	Architecture & Operational Capabilities
Core Framework	React 19 & TypeScript 5.9	Concurrent rendering, strict type contracts, and zero runtime layout shifts.
Client-Side Routing	TanStack Router	Type-safe, file-based routing (**`ui/app/routeTree.gen.ts`**) with deep URL state synchronization.
Global State Management	Redux Toolkit & RTK Query	Automated cache invalidation, optimistic UI mutations, and real-time polling.
Component Primitives	Radix UI & Tailwind CSS v4	Accessible unstyled primitives, high-contrast dark mode, and bespoke slate/emerald styling.
Code & Policy Editor	Monaco Editor (VS Code Engine)	Live syntax highlighting, auto-completion, and inline validation for CEL and JSON policies.
Data Visualization	Recharts & Lucide Icons	Real-time threat distribution charts, token burn gauges, and interactive latency histograms.
Binary Embedding	Go **`//go:embed`** Virtual FS	The compiled production UI is embedded directly into the Go binary; zero separate web server required.

Browser AI Dedicated Management Suite (`ui/app/workspace/browser-ai/`)

- **Overview Tab (`overview`):** Fleet-wide metrics, active workstation counts, 24-hour threat heatmaps, and top blocked domains.
- **Logs Tab (`logs`):** Full-text searchable audit log of every intercepted prompt, file attachment, risk score, and policy verdict.
- **Search Logs Tab (`search-logs`):** Search-engine queries and outbound link clicks captured across monitored browsers.
- **Rules Tab (`rules`):** Policy builder supporting regular expression patterns, arithmetic checksums, and AI Guard Bot evaluators.
- **Targets Tab (`targets`):** Domain governance catalog defining which AI sites are monitored, intercepted, or completely blocked.
- **Agents Tab (`agents`):** Live fleet inventory displaying hostname, OS, IP address, heartbeat status, and proxy bundle version.
- **Telemetry Tab (`telemetry`):** Real-time interception latency waterfall, CPU/memory usage, and branch network metrics.
- **Setup Tab (`setup`):** Download links for Windows Setup.exe and macOS PKG installers, and one-click proxy bundle re-publishing.

2.7 Complete HTTP API Surface & Route Catalog

The gateway binary exposes two distinct operational planes: the **`/v1/*`** data plane and the **`/api/*`** control plane:

Route Namespace	Key HTTP Endpoints	Authentication & RBAC Enforcement

Inference Data Plane	`POST /v1/chat/completions`
	`POST /v1/completions`
	`POST /v1/embeddings`
	`POST /v1/rerank`
	`POST /v1/ocr`
	`POST /v1/audio/transcriptions`
	`POST /v1/realtime`
		Authenticated via Virtual Key or Master Provider Key (`Authorization: Bearer <key>` or `x-api-key: <key>`). Enforces FinOps budget quotas and rate limits.

Provider Pass-Through	`POST /openai/*`
	`POST /anthropic/*`
	`POST /genai/*`
	`POST /bedrock/*`
	`POST /cohere/*`
	`POST /cursor/*`
		Direct native provider pass-through preserving vendor-specific headers and payload extensions while enforcing inline DLP.

Browser AI Control	`GET /api/browser-ai/logs`
	`GET /api/browser-ai/search-logs`
	`GET/POST /api/browser-ai/rules`
	`GET/PUT /api/browser-ai/controls`
	`GET /api/browser-ai/targets`
	`GET /api/browser-ai/agents`
		Gated by RBAC resource `Logs: View/Update` or section grant `browser-ai`. Intercept events authenticated via `UNIFAI_GUARD_SECRET`.

Guard Fleet Ingest	`POST /api/browser-ai/intercept`
	`POST /api/browser-ai/intercept-file`
	`POST /api/browser-ai/agents/heartbeat`
	`GET /api/browser-ai/proxy.pac`
		Secret-validated agent endpoints (`UNIFAI_GUARD_SECRET`). Fails closed if secret is missing or invalid (`UNIFAI_GUARD_REQUIRE_SECRET=1`).

Governance & FinOps	`GET/POST /api/governance/virtual-keys`
	`GET/POST /api/governance/teams`
	`GET/POST /api/governance/business-units`
	`GET/POST /api/governance/routing-rules`
		Administrative session cookie or API key. Enforces fine-grained Resource × Operation RBAC.

Workspace & Users	`GET/POST /api/users`
	`GET/PUT /api/roles`
	`GET /api/permissions`
	`GET/POST /api/access-profiles`
		Admin-only privileged routes. Audited automatically by `WorkspaceAuditMiddleware`.

Model Context Protocol	`GET /api/mcp/clients`
	`GET /api/mcp/library`
	`GET/POST /api/mcp/tool-groups`
	`POST /api/mcp/inference`
		Gated by `MCPGateway: View/Create/Update` permissions and per-user credential vault.

Health & Metrics	`GET /health`
	`GET /metrics`
	`GET /robots.txt`
	`GET /api/version`
		Unauthenticated public endpoints. `/health` performs concurrent pings to ConfigStore, LogsStore, and VectorStore.

2.8 Technical Extension Points Matrix

YP-Raksha is architected for frictionless enterprise customization. The following matrix guides engineering teams extending platform functionality:

Architectural Extension Goal	Primary Implementation Location	Integration Contract & Design Rules
Add a New LLM Provider	`core/providers/<vendor>/`	Create a new provider package implementing the **`Provider`** interface in **`core/schemas`**, then register in **`core/providers/registry.go`**.
Add a Pass-Through API	`transports/unifai-http/integrations/`	Implement the vendor HTTP bridge and register route prefix in **`integrations/router.go`**.
Add an Interception Plugin	`plugins/<plugin_name>/`	Implement **`BasePlugin`**, **`LLMPlugin`**, or **`HTTPTransportPlugin`**. Register in **`server/plugins.go`** (built-in) or compile as dynamic **`.so`**.
Add a Dashboard API Route	`transports/unifai-http/handlers/`	Create a new handler file with **`RegisterRoutes(router, middlewares...)`** and wire into **`server.RegisterAPIRoutes`**.
Gate a Route with RBAC	`framework/rbac/enforce.go`	Add the route mapping in **`PathRequirementFor(method, path)`** and configure section grants in **`framework/rbac/sections.go`**.
Add an RBAC Resource	`framework/configstore/workspace.go`	Add resource identifier to **`RBACResourceNames`**. All 6 operations (**`Read`**, **`View`**, **`Create`**, **`Update`**, **`Delete`**, **`Download`**) are generated automatically.
Add a UI Dashboard Page	`ui/app/workspace/<route>/`	Create **`page.tsx`** and **`layout.tsx`**, register the route in **`ui/lib/constants/workspaceSections.ts`**, and add navigation entry in **`sidebar.tsx`**.
Add a Guard Detection Rule	`apps/browser-guard/proxy/unifai_proxy_parts/`	Implement detection logic in the relevant proxy part, respecting the strict load order in **`MANIFEST.txt`**, and rebuild installer.
Add a New Guard Rule Type	`framework/logstore/browser_ai.go`	Update **`BrowserGuardRule`** GORM model, add frontend fields in **`guardRuleAIEvaluatorFields.tsx`**, and update proxy evaluator.
Customize Inline Guardrails	`plugins/guardrails/`	Extend inline inspection algorithms and configure validation policies via **`/api/guardrails/*`**.

2.9 Complete GORM Relational Database Schemas

The database schema is strictly normalized across PostgreSQL relational tables managed via automated GORM migrations:

Core Governance & ConfigStore Tables

- **`governance_users`:** Central user identity table (**`id`**, **`username`**, **`email`**, **`password_hash`**, **`role`**, **`status`**, **`budget_id`**, **`rate_limit_id`**, **`allowed_sections`**, **`reviewed_at`**).
- **`rbac_roles`:** Fine-grained role definitions (**`id`**, **`name`**, **`description`**, **`is_system_role`**, **`dac`** [**`all-data`** | **`own-data`**], **`permission_ids`** [JSON array]).
- **`access_profiles`:** Reusable enterprise governance policy templates (**`id`**, **`name`**, **`version`**, **`is_active`**, **`calendar_aligned`**, **`spec`** [JSON]).
- **`governance_virtual_keys`:** Cryptographic virtual credentials issued to applications (**`id`**, **`key_hash`**, **`name`**, **`user_id`**, **`team_id`**, **`customer_id`**, **`budget`**, **`rate_limit`**).
- **`governance_business_units`:** Departmental cost centers (**`id`**, **`name`**, **`team_ids`** [JSON], **`budget`** [JSON], **`rate_limit`** [JSON]).
- **`governance_routing_rules`:** Dynamic LLM model routing policies (**`id`**, **`name`**, **`priority`**, **`condition_expression`**, **`target_provider_id`**, **`target_model_id`**).

Browser AI & Workstation Fleet Tables (`framework/logstore/browser_ai.go`)

- **`browser_ai_logs`:** Intercepted prompt and file upload records (**`id`**, **`timestamp`**, **`platform`**, **`domain`**, **`user_prompt_full`**, **`action`**, **`status`**, **`rule_triggered`**, **`risk_score`**, **`predictive_risk`**, **`agent_id`**, **`agent_hostname`**, **`client_ip`**, **`attachment_name`**, **`attachment_size`**, **`attachment_path`**).
- **`browser_ai_search_logs`:** Search engine activity (**`id`**, **`timestamp`**, **`engine`**, **`browser`**, **`is_incognito`**, **`query`**, **`clicked_url`**, **`clicked_title`**, **`agent_id`**).
- **`browser_guard_rules`:** Screening rules (**`id`**, **`name`**, **`pattern`**, **`rule_type`** [**`regex`** | **`ai_bot`**], **`severity`**, **`action`** [**`BLOCK`** | **`REDACT`** | **`WARN`**], **`warning_message`**, **`active`**, **`bot_provider`**, **`bot_model`**, **`bot_prompt`**).
- **`browser_ai_target_websites`:** Monitored domain catalog (**`id`**, **`domain`**, **`platform_name`**, **`monitored`**, **`block_entire_website`**, **`host_role`** [**`ui`** | **`chat`** | **`file`**], **`status`** [**`MONITORED`** | **`PAUSED`** | **`BLOCKED`**]).
- **`browser_ai_control_settings`:** Interaction policies (**`id`**, **`allow_file_uploads`**, **`allow_file_downloads`**, **`warning_banner_text`**, **`enforce_strict_ssl`**).
- **`browser_ai_agents`:** Workstation fleet inventory (**`id`**, **`hostname`**, **`agent_type`** [**`endpoint`** | **`network`**], **`status`** [**`active`** | **`uninstall_pending`** | **`uninstalled`**], **`last_seen_at`**, **`health_detail`**, **`contact_email`**, **`uninstall_key_hash`**, **`uninstall_key_enc`**, **`proxy_bundle_sha`**).
- **`browser_ai_agent_settings`:** Master enterprise uninstallation secret (**`id`**, **`uninstall_key_enc`**).
- **`browser_guard_fleet_config`:** Company-wide default configurations pulled by agents on heartbeat (**`id`**, **`heartbeat_interval`**, **`pac_sync_interval`**).
- **`browser_guard_rebuild_logs`:** Audit trail of proxy package builds (**`id`**, **`status`**, **`bundle_sha`**, **`message`**, **`created_at`**).

- **`browser_ai_warning_emails`:** Outbound security notification log (**`id`**, **`sent_at`**, **`to_email`**, **`message`**, **`agent_id`**, **`error_detail`**).

2.10 Enterprise Cryptography, FIPS-140 Baseline & Security Model

YP-Raksha enforces a multi-tier defense-in-depth security model designed to satisfy stringent banking and defense baselines:

Security Dimension	Engineering Mechanism & Technical Implementation
FIPS-140 Baseline	All cryptographic operations execute via NIST-approved primitives (`crypto/tls`, `crypto/aes`, `crypto/sha256`). Master secrets derived via PKCS#11 HSM or HashiCorp Vault.
Secrets at Rest	Secrets stored in the database (provider master keys, webhook secrets, uninstall keys) are encrypted using AES-256-GCM via `framework/encrypt` when `UNIFAI_ENCRYPTION_KEY` is configured.
Session Security	Dashboard sessions use cryptographic UUIDs stored in the database, delivered via `HttpOnly`, `SameSite=Lax`, and `Secure` cookies with 24-hour expiration TTL.
WebSocket Security	WebSockets reject URL query token authentication to eliminate token leaks in logs; short-lived tickets are issued via authenticated `POST /api/session/ws-ticket`.
Brute-Force Lockout	Per-IP rate limiting (maximum 30 failed login attempts per 15-minute sliding window) combined with per-username database lockouts.
Password Reset Security	HMAC-SHA256 signed tokens using `PASSWORD_RESET_SECRET` (minimum 32 characters) with 10-minute validity, strict 5 requests/hour limits, and 2-minute cooldowns.
Fleet Mutual Trust	Workstation agents authenticate to backend APIs using `UNIFAI_GUARD_SECRET`. When `UNIFAI_GUARD_REQUIRE_SECRET=1`, all unauthenticated agent requests fail closed.
PAC Anti-Hijack	By default, `UNIFAI_PAC_ALLOW_QUERY_PROXY=0` forces the PAC script to serve only administrator-approved proxy IP addresses, completely preventing PAC-redirection attacks.
Anti-Tamper Uninstallation	Workstation agent uninstallation requires a 32-character enterprise uninstallation key verified against a salt-hashed SHA-256 digest in the central database.

2.11 Production Environment Variables & Deployment Parameters

Environment Variable	Type / Default	Operational Purpose & Architectural Impact
`APP_PORT` / `-port`	Integer (`8001`)	Primary TCP listening port for the unified Go HTTP server.
`APP_HOST` / `-host`	String (`localhost`)	Network interface binding (`0.0.0.0` binds to all network interfaces).
`APP_DIR` / `-app-dir`	Path (`./data`)	Local directory path for file attachments, offline buffers, and SQLite databases.
`UNIFAI_DB_TYPE`	String (`postgres`)	Persistence engine selection: `postgres`, `mysql`, or `sqlite`.
`UNIFAI_DB_DSN`	Connection String	Database Data Source Name (e.g. `postgresql://user:pass@host:5432/raksha_db?sslmode=verify-full`).
`UNIFAI_ENCRYPTION_KEY`	Hex String (64 chars)	256-bit AES cryptographic master key used to encrypt database secrets at rest.
`UNIFAI_GUARD_SECRET`	String (Alphanumeric)	Shared secret used to authenticate workstation Guard agents to the `/api/browser-ai/*` backend.
`UNIFAI_GUARD_REQUIRE_SECRET`	Boolean (`1`)	When set to `1`, forces all agent ingest endpoints to fail closed if the secret is missing or invalid.
`UNIFAI_PAC_ALLOW_QUERY_PROXY`	Boolean (`0`)	When set to `0`, prevents clients from overriding the PAC proxy IP via URL query parameters.
`SERVER_DOMAIN`	String (FQDN)	Fully qualified domain name used to configure CORS allowed origins and generate PAC URLs.
`PASSWORD_RESET_SECRET`	String (Min 32 chars)	Secret key used to sign HMAC tokens for self-service password reset workflows.
`LOG_LEVEL`	String (`info`)	Logging verbosity: `debug`, `info`, `warn`, `error`. Production default is `info`.

2.12 Developer, Build & Automated Test Workflow

The repository includes a comprehensive `Makefile` automating the entire continuous integration, containerization, and distribution pipeline:

```
# Gateway & Core Development Workflow
make setup-workspace     # Initializes the multi-module Go workspace (go work use ./core ./framework ...)
make dev                 # Concurrently runs Vite UI and Air live-reloading Go backend
make build               # Builds the production React UI and compiles the hermetic Go binary
make fmt ; make lint     # Enforces gofmt code styling and oxlint/golangci-lint quality gates

# Automated Verification & Test Suite
make test                # Executes unit tests for unifai-http transport via gotestsum
make test-core           # Tests core provider adapters (PROVIDER=openai TESTCASE=...)
make test-framework      # Runs relational persistence and RBAC verification tests
make test-plugins        # Validates plugin pipeline ordering and latency budgets
make test-all            # Executes the complete unit, integration, and performance test suite
make run-e2e             # Runs Playwright browser end-to-end automation test suite

# Workstation Guard Packaging Workflow
make sync-guard-config   # Synchronizes agent JSON configurations and documentation from .env
make build-guard-windows # Compiles Windows executable and packages Inno Setup 6 Setup.exe installer
make build-guard-mac     # Packages macOS Application Bundle (.app) and signed PKG installer
```

2.13 Enterprise SBOM, Third-Party Library Governance & SLSA Level 3

YP-Raksha adheres to Supply-chain Levels for Software Artifacts (SLSA) Level 3 security standards:

- **Hermetic & Reproducible Builds:** Gateways and host agents are compiled in fully isolated environments with pinned toolchains (**`CGO_ENABLED=0`**, **`-trimpath`**, **`-buildvcs=false`**). The resulting binary is bit-for-bit reproducible.
- **Cryptographic Provenance Attestation:** Every build generates an in-toto attestation signed with the bank's internal PKI or Cosign cryptographic keys, capturing the exact commit SHA, build environment, and SHA-256 hashes of all input sources.
- **Automated Daily SCA & CVE Auditing:** Automated pipelines scan the codebase daily using Syft, Grype, and Trivy against the NIST National Vulnerability Database (NVD). Releases with unresolved CVSS scores >= 7.0 are automatically rejected.
- **Tamper-Evident Delivery:** Artifacts are delivered alongside GPG signatures and SHA-256 checksums, verified by the bank's automated deployment pipelines prior to staging.